

Measuring the Security of the Evolving Software Supply Chain: a Research Agenda

Sarah Meriem Ourari

CrySyS Lab

Budapest Univ. of Technology and Economics, Hungary

sarah.ourari@crysyst.hu

Abstract—Software supply chain security has become increasingly critical due to the widespread reliance on third-party dependencies and the growing attack surface of modern software ecosystems. However, existing quantitative, measurement-based analysis and vulnerability management approaches remain largely fragmented and ecosystem-specific, limiting their ability to provide comparable risk assessments across environments.

This paper presents a structured research plan, starting with a Systematization of Knowledge (SoK) to synthesize the current state of research and identify key gaps, highlighting the limitations in dependency modeling and vulnerability propagation analysis, particularly in the treatment of transitive dependencies and their real-world exploitability. Based on these insights, we argue for a unified measurement perspective capable of consistently representing and analyzing the cross-ecosystem dependency structure. We further identify emerging challenges introduced by AI-assisted software development, where coding LLMs are likely to contribute to new dependency patterns that are not captured by traditional Software Composition Analysis (SCA) tools. These shifts motivate a rethink of dependency modeling to account for evolving software-generation practices and their long-term structural impact on software security.

Index Terms—supply chain security, software security, vulnerability exposure

I. INTRODUCTION

A software supply chain encompasses the entire end-to-end lifecycle of an application, from upstream development stages to downstream operational phases, as illustrated in Figure 1. Securing this pipeline requires enforcing continuous security practices, such as secure coding, vulnerability scanning, and dependency validation, to protect both code generation and artifact delivery from malicious threats.

The importance of software supply chain security has grown significantly in recent years. Supply chain attacks have demonstrated that the compromise of a single dependency or maintainer can have widespread consequences across thousands of downstream applications. As a result, understanding and securing dependency relationships has become a central concern in both software engineering and cybersecurity research.

Despite this growing attention, several challenges remain unresolved. Security risks propagate not only through direct dependencies but also through transitive dependency chains, making vulnerability tracking highly complex. In addition, SCA tools often produce false positives, suffer from inaccurate dependency resolution, and fail to properly model real-world execution and reachability [1]. Moreover, many existing approaches are fragmented and tailored to specific ecosystems,

limiting their generalizability and making cross-ecosystem comparisons and risk assessments difficult.

In parallel, the increasing adoption of automated software development raises new questions for software supply chain security. Beyond traditional dependency management challenges, AI-generated code may introduce novel dependency relationships, influencing how developers select, reuse, and maintain software components. As a result, existing measurement and dependency modeling methodologies may need to be adapted/extended to accurately capture these emerging behaviors and their impact on software ecosystems.

In this context, this paper outlines future research aimed at developing a generalized framework for systematically measuring software security across multiple software ecosystems. This study is driven by the need to bridge existing methodological gaps and explore how AI-assisted code generation affects ecosystem-level security analysis. Based on the identified limitations and gaps, this work addresses the following research questions:

- **RQ0: What is the current state of the art in measuring and modeling software supply chain security across different software ecosystems?**
- **RQ1: How can a unified measurement model be designed to analyze dependency graphs and assess vulnerability exposure across diverse software ecosystems?**
- **RQ2: How can source code-level reachability analysis reduce alert fatigue by identifying which vulnerabilities in transitive dependencies are actually executable?**
- **RQ3: To what extent does LLM-generated code introduce new forms of software dependencies, and how can the proposed measurement and dependency modeling framework be extended to capture them?**
- **RQ4: How have dependency patterns evolved across software ecosystems following the adoption of LLM-assisted software development compared to the pre-LLM era?**

This paper is organized as follows. Section II presents a condensed systematic literature review on measurable software supply chain security. Section III provides a synthesis of the main research gaps identified across multiple software ecosystems. Section IV outlines a future research agenda, while Section V concludes the paper.

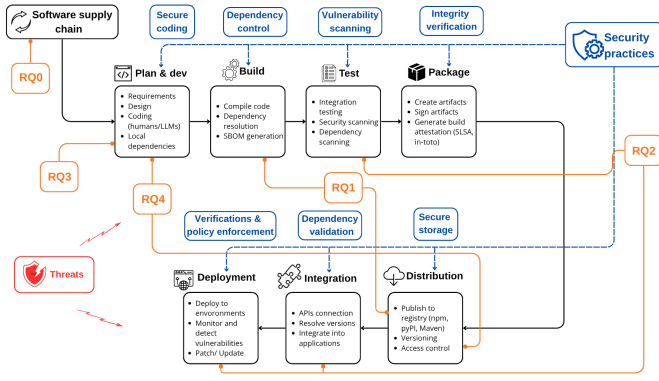


Fig. 1. The Software Supply Chain Lifecycle and security mapping framework

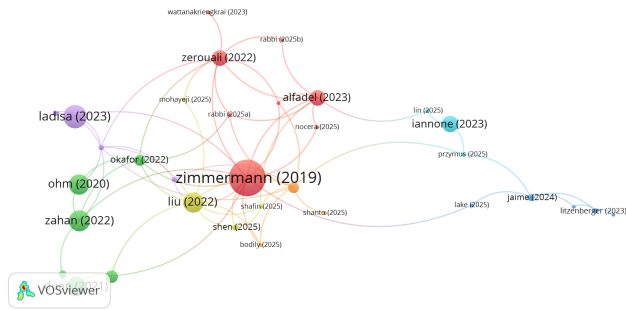


Fig. 2. Citation network and clustering visualization using VOSviewer

II. STATE OF THE ART¹

A structured literature review methodology was followed to collect and analyze the relevant works in the field of software supply chain security. The articles were collected from scientific databases such as IEEE Xplore, ACM Digital Library, SpringerLink, Google Scholar, and Web of Science, with a focus on recent publications to ensure up-to-date findings. The selection focused on papers relevant to quantifiable software ecosystem security, dependency analysis, and vulnerability management, excluding non-scientific sources and low-quality papers.

After filtering the articles, we constructed citation graphs using bibliometric analysis tools, e.g., VOSviewer and CiteNet Explorer, to visualize relationships among papers, identify influential works, and analyze citation patterns across studies. The resulting graph generated with VOSviewer is shown in Figure 2.

Lastly, the selected literature was organized into thematic categories, including ecosystem-based and software metrics studies, SCA tools, vulnerability mitigation and patching techniques, and dependency graph structure and management. As part of the findings, several key results can be summarized as follows.

¹a full-length Systematization of Knowledge (SoK) paper is under submission

A. Ecosystem risk dynamics and dependency metrics

To map the global risk environment, researchers focused on quantifying how architectural choices and ecosystem management structures influence overall ecosystem risk. Various studies were conducted to identify major factors, assess the spread of vulnerabilities, and determine the real impact of dependency maintenance on ecosystem security. Zimmermann et al. [2] investigated the influence of packages and maintainer accounts in compromising large parts of the ecosystem, focusing on direct dependencies, through reachability analysis and code vetting. Similarly, Zahan et al. [3] empirically measured *npm* weak links, enhancing previous metrics by introducing trust scores to capture vulnerability influence and by modeling collusion attacks to better characterize maintainer impact, highlighting both direct and transitive dependencies.

Taken together, these studies converge on two key structural patterns in software ecosystems: i) popular packages have a high-impact attack surface, as several libraries depend on them, and ii) a small number of highly influential maintainers affect a large portion of the ecosystem.

The study by Zahan et al. [3] also reported several supply chain attack vectors, including install scripts, unmaintained packages, and expired domains, arising from maintainers being overloaded with a large number of packages. To address these maintenance bottlenecks, Rahman et al. [4] emphasized the importance of dependency security by introducing two new metrics: $MTTRdep^2$ and $MTTDep^3$. Their quantitative results indicate that the effectiveness of dependency updates reflects how well a package addresses vulnerabilities, suggesting that improving update practices can significantly strengthen overall security.

Beyond structural dependency risks, researchers have also investigated how vulnerabilities evolve and are managed across different software ecosystems. Alfadeli et al. [5] studied the *PyPI* ecosystem in comparison with *npm* to understand its specific characteristics, while also developing DEPHEALTH, a tool designed to detect and report vulnerabilities. On the other hand, Zerouali et al. [6] examined both *npm* and *RubyGems* ecosystems. Synthesizing findings across these environments, both studies reported that most discovered vulnerabilities are of medium or high severity, with Cross-Site Scripting (XSS) being the most common vulnerability type. Another critical cross-ecosystem finding concerns the differences in disclosure speed. For instance, *npm* appears to have faster patching of identified vulnerabilities, but may still leave some issues unresolved for longer periods due to unknown real discovery dates and a lack of CVE identifiers. In contrast, *PyPI* tends to have longer update delays, as vulnerabilities are disclosed more quickly, reducing the remediation window compared to *npm*. Despite these operational differences, both studies emphasize the role of indirect dependencies in increasing exposure, highlighting the importance of timely updates.

²MTTRdep (Mean Time To Remediate dependencies): Average time required to remediate vulnerable dependencies.

³MTTDep (Mean Time To Update dependencies): Average time required to update dependencies.

Finally, focusing on the *Maven* ecosystem, Rabbi et al. [7] and Nachuma et al. [8] analyzed vulnerability evolution and dependency management processes using the Goblin framework [9] and the CWE database. These studies concluded that most vulnerabilities stem from recurring software weaknesses, particularly issues in input validation and access control, unencrypted data handling, and authentication failures. In addition, the patching process was found to be relatively slow, highlighting the need for faster vulnerability discovery, reporting, and documentation.

B. Software Composition Analysis and reachability

In order to analyze software dependencies and detect these security vulnerabilities, SCA tools are commonly used in research. Studies revealed several limitations in their ability to handle the complexity of software ecosystems, as most of them focus only on direct dependencies while ignoring dependency resolution issues, leading to false vulnerability warnings and reduced accuracy. Zhao et al. [1] investigated the effectiveness of SCA tools in the Java *Maven* ecosystem by proposing a new evaluation model that considers execution environments, dependency and vulnerability detection mechanisms, and dependency selection. Results show difficulties in handling cross-project information, such as parent configurations and third-party component management, leading to missing inherited packages or incorrect versions in pre-built scans. In addition, there is no distinction between the types of external libraries. As a result, system dependencies are integrated into the software, increasing false positives. The study also revealed that vulnerability detection performance heavily relies on dependency analysis. Therefore, research confirms the need to use the developed Scan Scope Maven (SSM) concept, which reduces both false positives and false negatives by defining the required dependencies to include during project analysis. Furthermore, researchers highlight the need for SCA tools to extend detection beyond package managers to include external references and copy-pasted code [1], which represent a large part of the *Maven* ecosystem, and to choose the tools based on the use case since the execution environment considerably influences the results. These limitations have also been observed across other SCA tools and ecosystems, suggesting that the issue is not tool-specific but fundamentally related to dependency resolution and reachability modeling.

Building on those observations, subsequent work has examined SCA tools in real-world settings, focusing on how vulnerability detection outcomes influence developers' decision-making. Nocera et al. [10], through their cohort study, confirm previous findings regarding the suitability of the OWASP Dependency-Check tool for built projects, with results showing a considerable reduction in the number and severity of vulnerabilities, especially high-severity and informational vulnerabilities. Similarly, Mohajeji et al. [11] extended this research by investigating the practical usefulness of Dependabot through an analysis of developers' reactions to its security updates in the *npm* ecosystem. Results show that in more than half of the cases, developers directly apply the fixes suggested by

Dependabot without observing major issues after integration. In addition, the study revealed that automated fixes are more frequent than manual ones, except for major version upgrades due to compatibility constraints. Additional improvements suggest incorporating vulnerability reachability analysis, as some detected weaknesses may not be reached and therefore may not represent relevant threats. This could help reduce unnecessary alerts and decrease false positives.

To improve the precision of vulnerability detection, recent research has shifted toward reachability analysis and graph-based dependency modeling techniques. Wu et al. [12] explored source code-level analysis to better detect actual risks. The study shows that most identified vulnerable functions do not pose a real danger in practice, as only a small percentage is reachable, and many vulnerabilities are either deeply embedded in the dependency tree or require complex conditions to be exploited. Therefore, SCA tools frequently raise alerts even when there is no real danger, overloading developers with unnecessary updates. An additional factor contributing to the inaccuracy of SCA tools is the presence of ecosystem-specific characteristics. Liu et al. [13] developed a solution to overcome limitations of *npm* rules by constructing a knowledge graph linking packages to their known vulnerabilities, along with an algorithm to accurately reconstruct the true dependency graph, achieving more precise results.

C. Vulnerability remediation and lifecycle principles

To address challenges in vulnerability management, Iannone et al. [14] emphasized the importance of extending existing analysis tools by incorporating development timelines, developer actions, and contextual changes into vulnerability assessment. Similarly, Okafor et al. [15] introduced three key security principles combining dependency transparency, component validity, and the separation of development stages to support ecosystem-wide analysis.

Building on these conceptual foundations, several studies have proposed automated approaches to improve vulnerability remediation in practice. Zhang et al. [16] introduced Ranger, a tool that automatically restores version ranges while preserving compatibility with existing code bases, and thus, reducing the risk of breaking changes and limiting the persistence of security flaws. However, accurately linking vulnerabilities to their corresponding fixes remains a challenging problem. Li et al. [17] address this issue with PatchFinder, a system that identifies relevant patch commits by leveraging semantic similarity and contextual code understanding, thereby improving the reliability of the results.

D. Dependency graph construction and mining frameworks

Dependency graphs, as one of the primary artifacts used in software ecosystem analysis, have received considerable attention due to the growing complexity of software supply chains and the increasing number of attacks. To address the lack of standardization in dependency graph construction, Litzenberger et al. [18] proposed the Dependency Graph Mining Framework (DGMF) as a unified solution for building

dependency graphs from different software repositories. The results demonstrated improved execution efficiency through reduced runtime and simplified adaptation to heterogeneous repositories. However, a trade-off still occurs during dependency resolution, which comes at the cost of increased computational efforts.

Building on the need for more accurate dependency representations, Jaime et al. [9] developed the Goblin framework to support historical analysis. The framework reconstructs dependency graphs while incorporating temporal information, enabling more precise analysis and on-demand computation of ecosystem metrics, which improves dependency quality. Their findings revealed several factors that can influence research outcomes, including differences in dataset size and structure, dependency updates and management, and the high computational costs associated with metric processing across transitive dependencies.

Beyond the technical challenges of dependency graph construction and analysis, researchers have also identified inefficiencies in the research process. Many studies repeatedly rebuild the same basic custom tools to collect and process ecosystem data, leading to duplicated effort, limited reusability, and a large number of abandoned or unmaintained research artifacts. To address these challenges, Dusing et al. [19] proposed the MARIN framework, which automates repetitive technical tasks in *Maven* ecosystem analysis, while improving performance through parallel processing. Similar efforts have also been undertaken in other software ecosystems. Filgueira et al. [20] developed the static analysis framework *inspect4py* for Python, which examines source code structure and extracts relevant software artifacts to support software engineering and ecosystem analysis tasks.

These findings reflect the current state of problems and potential solutions aimed at improving measurable aspects of software supply chain security; however, a lack of generalizability is evident. Most proposed solutions are ecosystem-specific and rely on particular characteristics, making comparison across studies difficult. This highlights the need for more comprehensive metrics for mitigation and management processes.

III. LITERATURE SYNTHESIS AND RESEARCH QUESTIONS

Based on previous studies, several important findings have contributed to a better understanding of software security chains. At the same time, the adoption of Large Language Models (LLMs) has introduced new challenges and transformations in software development and security analysis, highlighting the need for further investigations into their impact.

Although existing solutions make valuable contributions to vulnerability management and software security, most proposed approaches and metrics remain ecosystem-specific and rely on specific characteristics. This lack of generalization limits the development of unified mitigation and management strategies by centralizing research efforts on analyzing individual environments in isolation. As a result, there is no guarantee that approaches effective in one ecosystem can be

directly transferred to another. Therefore, the evaluation of such solutions remains constrained to the datasets and the conditions of a single ecosystem. Furthermore, this fragmented perspective limits the exploration of structural similarities and differences across ecosystems, potentially leading to an incomplete understanding of the global problem space. Based on these observations, we formulate **RQ1: How can a unified measurement model be designed to analyze dependency graphs and assess vulnerability exposure across diverse software ecosystems?**

In addition, most studies focus heavily on direct dependencies while oversimplifying the impact of transitive and third-party packages, which have a significant influence on ecosystem security. A common operational limitation is the difficulty of correctly resolving dependency versions, often driven by developers' fear of introducing breaking updates. Furthermore, not all vulnerabilities present in a dependency tree are necessarily reachable during program execution. As a result, developers are overwhelmed with excessive updates generated by SCA tools, which often miss certain packages or introduce unrelated dependencies into the analysis [1], reducing the credibility of the findings due to inaccuracies in the extracted graphs. This highlights an urgent need for more fine-grained analysis that can distinguish which updates are truly necessary and have a meaningful impact on ecosystem security. Such improvement can be achieved through source code-level reachability analysis to identify exploitable vulnerabilities and support more relevant dependency updates. Existing tools such as *CodeQL*⁴ illustrate how source-code analysis can be used to determine whether vulnerable code paths are actually reachable during execution. In addition, this perspective should also account for cross-project reuse and code cloning, as they represent a significant portion of software ecosystems. Based on these observations, we formulate **RQ2: How can source-code-level reachability analysis reduce alert fatigue by identifying which vulnerabilities in transitive dependencies are actually executable?**

While optimizing traditional dependency mapping represents a crucial foundational step, the modern software supply chain is facing an unpredictable paradigm shift due to the rapid integration of artificial intelligence in software development. The widespread adoption of LLMs for automated code generation has changed how applications are written, directly impacting how dependencies are introduced. Package recommendations generated by AI models may introduce distinct forms of software dependencies and hidden security risks at the source level. It is therefore critical to investigate how the foundational measurement and graph modeling frameworks must evolve to effectively capture this new class of supply chain elements. This leads to the next research question, **RQ3: To what extent does LLM-generated code introduce new forms of software dependencies, and how can the proposed measurement and dependency modeling framework be**

⁴static analysis engine developed by Github for querying source code as a database.

extended to capture them? In parallel, as AI-based tools become integrated into development workflows, both the volume of generated code and the practices surrounding code reuse are evolving. Understanding the implications of this shift requires comparing dependency structures before and after the widespread use of LLMs to reveal how dependency relationships, depth, and versioning practices have evolved over time. Consequently, this study seeks to quantify this historical transition by tackling **RQ4: How have dependency patterns evolved across software ecosystems following the adoption of LLM-assisted software development, compared to the pre-LLM era?**

IV. PROPOSED RESEARCH METHODOLOGY

The adopted methodology structuring this research can be summarized in three main stages: literature review and state of the art, the design of a generalized measurement approach, and the investigation of its applicability for improving software supply chain security. The overall workflow of the proposed methodology is illustrated in Figure 3.

A. Systematic literature review

The first phase consists of an analysis of existing studies on software system security, dependency management, and vulnerability mitigation, culminating in the writing of a SoK paper to understand the current state of research and analyze existing tools and metrics. From this literature overview, key limitations and research gaps are identified to highlight shortcomings in current approaches, leading to the formulation of the main research questions, which focus on the need for a more generalizable framework for ecosystem-level analysis.

B. Data collection and ecosystem extraction

This phase focuses on building a large-scale dataset of software ecosystems to support empirical analysis. Data is collected from multiple sources, including package managers (e.g., *npm*, *PyPI*, *Maven*), source code repositories such as *GitHub* and *Bitbucket*, and dependency metadata APIs. The collected data includes dependency graphs, version histories, release metadata, and vulnerability reports. The extracted datasets are then used to construct heterogeneous dependency networks across ecosystems, enabling both static and temporal analysis of software supply chains.

C. Design of a generalized measurement approach

The core contribution of this research lies in exploring the feasibility of a unified measurement model that captures and enables the evaluation of dependency-related risks and ecosystem-level security exposure across different software environments.

First, a unified dependency graph representation is developed to capture direct and transitive relationships across heterogeneous ecosystems. Based on this representation, a measurement framework is defined to quantify vulnerability exposure, dependency risk propagation, and ecosystem-level security impact.

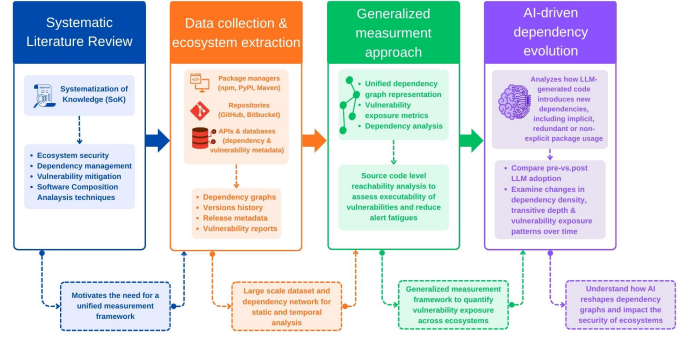


Fig. 3. Main phases of the proposed research methodology

To refine the vulnerability assessment, a source-code-level reachability analysis is integrated to determine whether vulnerabilities in transitive dependencies are actually executable in practice. This mechanism aims to reduce alert fatigue by distinguishing between theoretical exposure and real exploitability.

D. LLM-driven code generation and ecosystem evolution analysis

The last phase focuses on the impact of LLM-assisted software development code on dependency ecosystems. First, the study analyzes how LLM-generated code introduces new dependencies, including implicit, redundant, or non-explicit package usages. The previously defined measurement and modeling framework is extended to capture these newly emerging dependency patterns, potentially based on probabilistic graphical models [21]. Second, a temporal analysis is conducted to compare dependency structures before and after the widespread adoption of LLM-assisted development tools. This includes examining changes in dependency density, transitive depth, and vulnerability exposure patterns across ecosystems over time.

V. CONCLUSION

This paper presents a research plan based on the current state of the literature, derived from a SoK paper on quantifiable software supply chain security, covering ecosystem security, dependency management, vulnerability mitigation techniques, and SCA tools.

The review highlights the absence of a generalized measurement framework for consistent ecosystem-level security evaluation, motivating a research direction focused on the design of a unified software supply chain security measurement framework across multiple ecosystems, as well as an investigation into how artificial intelligence and LLMs reshape software dependency structures.

REFERENCES

- [1] L. Zhao, S. Chen, Z. Xu, C. Liu, L. Zhang, J. Wu, J. Sun, and Y. Liu, "Software composition analysis for vulnerability detection: An empirical study on java projects," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 960–972. [Online]. Available: <https://doi.org/10.1145/3611643.3616299>
- [2] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," *arXiv preprint arXiv:1902.09217*, 2019. [Online]. Available: <https://arxiv.org/abs/1902.09217>
- [3] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, 2022, pp. 331–340. [Online]. Available: <https://doi.org/10.1145/3510457.3513044>
- [4] I. Rahman, R. Paramitha, W. Enck, and L. Williams, "How quickly do development teams update their vulnerable dependencies?" *arXiv preprint arXiv:2403.17382*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2403.17382>
- [5] M. Alfadhel and D. E. Costa, "Empirical analysis of security vulnerabilities in python packages," *Empirical Software Engineering*, vol. 28, no. 1, pp. 1–34, 2023. [Online]. Available: <https://doi.org/10.1007/s10664-022-10278-4>
- [6] A. Zerouali, T. Mens, A. Decan, and C. D. Roover, "On the impact of security vulnerabilities in the npm and rubygems dependency networks," *Empirical Software Engineering*, vol. 27, no. 5, 2022. [Online]. Available: <https://doi.org/10.1007/s10664-022-10154-1>
- [7] M. F. Rabbi, R. Paul, A. I. Champa, and M. F. Zibran, "Understanding software vulnerabilities in the maven ecosystem: Patterns, timelines, and risks," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 290–294. [Online]. Available: <https://doi.org/10.1109/MSR66628.2025.00052>
- [8] C. Nachuma, M. M. Hossan, A. K. Turzo, and M. F. Zibran, "Decoding dependency risks: A quantitative study of vulnerabilities in the maven ecosystem," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 270–274. [Online]. Available: <https://doi.org/10.1109/MSR66628.2025.00048>
- [9] D. Jaime, J. E. Haddad, and P. Poizat, "Goblin: A framework for enriching and querying the maven central dependency graph," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 37–41. [Online]. Available: <https://doi.org/10.1145/3643991.3644879>
- [10] S. Nocera, S. Vegas, G. Scanniello, and N. Juristo, "Software composition analysis and supply chain security in apache projects: An empirical study," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 103–115. [Online]. Available: <https://doi.org/10.1109/MSR66628.2025.00027>
- [11] H. Mohayjei, A. Agaronian, E. Constantinou, N. Zannone, and A. Serebrenik, "Securing dependencies: A comprehensive study of dependabot's impact on vulnerability mitigation," *Empirical Software Engineering*, vol. 30, no. 3, p. 89, 2025. [Online]. Available: <https://doi.org/10.1007/s10664-025-10638-w>
- [12] Y. Wu, Z. Yu, M. Wen, Q. Li, D. Zou, and H. Jin, "Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1046–1058. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00095>
- [13] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 672–684. [Online]. Available: <https://doi.org/10.1145/3510003.3510142>
- [14] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, "The secret life of software vulnerabilities: A large-scale empirical study," *IEEE Transactions on Software Engineering*, vol. 49, no. 1, pp. 44–63, 2023. [Online]. Available: <https://doi.org/10.1109/TSE.2022.3140868>
- [15] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "Sok: Analysis of software supply chain security by establishing secure design properties," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2022, pp. 15–24. [Online]. Available: <https://doi.org/10.1145/3560835.3564556>
- [16] L. Zhang, C. Liu, S. Chen, Z. Xu, L. Fan, L. Zhao, Y. Zhang, and Y. Liu, "Mitigating persistence of open-source vulnerabilities in maven ecosystem," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 191–203. [Online]. Available: <https://doi.org/10.1109/ASE56229.2023.00058>
- [17] K. Li, J. Zhang, S. Chen, H. Liu, Y. Liu, and Y. Chen, "Patchfinder: A two-phase approach to security patch tracing for disclosed vulnerabilities in open-source software," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 590–602. [Online]. Available: <https://doi.org/10.1145/3650212.3680305>
- [18] T. Litzenberger, J. Düsing, and B. Hermann, "Dgmf: Fast generation of comparable, updatable dependency graphs for software repositories," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 115–119. [Online]. Available: <https://doi.org/10.1109/MSR59073.2023.00028>
- [19] J. Düsing, J. Chiaramonte, and B. Hermann, "Marin: A research-centric interface for querying software artifacts on maven repositories," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025, pp. 591–595. [Online]. Available: <https://doi.org/10.1109/MSR66628.2025.00093>
- [20] R. Filgueira and D. Garijo, "Inspect4py: A knowledge extraction framework for python code repositories," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 232–236. [Online]. Available: <https://doi.org/10.1145/3524842.3528497>
- [21] D. Koller and N. Friedman, *Probabilistic graphical models: principles and techniques*. MIT press, 2009.